15-Feb-2026

Date
Page

Javascript Essentials Const.

0.7- array

How to design array.

Const carriage1 = ["Veer", "Aysh", "Ravi"]

Const emptyCarriage = [] ⇒ it's print's the [] empty array.

Const threeEmptySeats = Array (3) — array constructor

Console.log (threeEmptySeats);

⇒ [<3 empty items>] (Kuchh nahi hai khali jagah hai)

Console.log (threeEmptySeats.length);

⇒ 3 (3 empty items)

Const passenger = Array ("Veer", "Ayush", "Ravi");

→ both have some output

['Veer', 'Aysh', 'Ravi']

Const singlePassenger = Array.of (3);

Console.log (singlePassenger); ⇒ [3]

What is the difference between Array (3) and Array.of(3)

Array (3) :- when you pass the 3 number in the Array () constructor then Js treats the number as the length of the array, not a first element.

- It print's 3 empty items but no actual element.
- and the length of this array is 3

Array.of (3):- It creates the array ~~of 3~~ where the arguments you provide 3 to become the actual elements.

- It print's 3 the actual element.
- and the length of this array is 1.

Note :- Empty slots it not a undefined or not a null in array

Create array using different values
(ex, strigs, sets or the objects)

- Array.form()

It create a new, shallow-copied Array instance from an array-like or interable object. This method is used to convert various data types into a standard javascript array.

ex.

```
Const tempTrain = TrainCode = Array.from("DUST")
Console.log(TrainCode);
```

⟹ ['D', 'U', 'S', 'T']

- Truncate of array

truncad an array mems to shorten it's length, which either mutate the original array or return a new, truncated array

ex.

```
Const tempTrain = ["A", "B", "C", "D", "E"];
tempTrain.length = 3;
Console.log(tempTrain);
```

⟹ ["A", "B", "C"] ("D", "E" data lost)

define length again

```
tempTrain.length = 5;
Console.log(tempTrain);
```

lost data

⟹ ["A", "B", "C", <2 empty items>]

Note :- This is not good think to overwrite the property don't use this method to overcorite the array property

- array methods

* Mutating array methods

- push()

this method is added one or more elements to the end of an array and modifies the original array in place

ex.

```
Const fruits = ["apple", 'banana"]
fruits.push("Orange");
Console.log(fruits);
```

=> ['apple', 'banana', "Orange"]

- pop()

this method is used to remove the last element from an array and return that removed element.

ex.

```
Const fruits = ["apple", "banana", "Orange"]
Const removefruits = fruits.pop();
Console.log(fruits);
```

=> ['apple', 'banana'] (original array is changed)

```
Console.log(removefruits);
```

=> orange (removed element)

- shift()

this method is used to remove the first element from an array.

ex.

```
Const animals = ["Dog", "Cat", 'Rabbit"]
animal.shift();
Console.log(animals);
```

=> ~~["Dog", "Cat"]~~ => ['Cat', 'Rabbit']

- unshift() (VIP)

This method is used to add one or more elements to the beginning of an array.

ex.

```
const fruits = ['Apple', 'Orange']
fruits.unshift('Banana');   ← add element first
console.log(fruits);
```

=> ['Banana', 'Apple', 'Orange']

- splice()

This method is for modifying an array in place by adding, removing, or replacing elements.

1) Deleting elements

- to delete one or more elements, specify the start index and the number of elements to delete count

ex.

```
const fruits = ['apple', 'banana', 'cherry', 'date'];
                 0        1         2         3
fruits.splice(1, 1);
              start end
// Remove 1 element starting from index 1
console.log(fruits);
```

=> ['apple', 'cherry', 'date'] (removes the banana)

2) Adding Elements

To add elements without deleting any, set deleteCount to 0 and provide the new items

ex

```
const fruits = ['apple', 'banana', 'date']
                 0        1         2
fruits.splice(2, 0, 'cherry', 'elderberry');
// add elemend staring from index 2
```

Two element add

Console.log (fruits)
⇒ ['apple', 'banana', 'cherry', 'elderberry', 'date']

3) Replacing Elements
To replace elements, specify both the deleteCount of existing elements to remove and the new items to insert
ex,

```
Const fruits = ['apple', 'banana', 'cherry', 'date']
fruits.splice (1, 2, 'grape')
// replace 2 elements starting from index 1
('banana', 'cherry' with 'grape')
Console.log (fruits)
→ ['apple', 'grape', 'date']
```

* Unmutating array methods (important in React redux)
(returns new array without touching original)

1) Concat () (concat ⇒ Concatenate (connect))
This method that can merge two or more arrays. and it does not change the existing arrays. but instead returns a new array.

ex,
- merge two arrays

```
Const array1 = [1, 2, 3];
Const array2 = [4, 5, 6];

Const mergedArray = array1.concat (array2)
Console.log (mergedArray)
⇒ [1, 2, 3, 4, 5, 6]
```

merged multiple arrays

```
Const Array1 = [1, 2, 3]
Const Array2 = [4, 5, 6]
Const Array3 = [7, 8, 9]

Const mergedArray = Array1.concat(Array2, Array3);
Console.log(mergedArray).
```

(Separated by commas)

⇒ [1, 2, 3, 4, 5, 6, 7, 8, 9]

merged with individual value

```
Const Array1 = [1, 2, 3]

Const individualValue = Array1.concat(4, 5, 6);
Console.log(individualValue);
```

⇒ [1, 2, 3, 4, 5, 6]

2) Slice ()

This method in array returns shallow copy of a portion of an array into a new array object selected from start to end, start and end are the index of array.

ex,

```
Const animals = ["ant", "Camel", "Dog", "Rabbit"]
Console.log(animals.slice(2))
```

⇒ ['Dog', 'Rabbit']

```
Const animals = ["ant", "Camel", "Dog", "Cat", "Rabbit"]
Console.log(animals.slice(2,4))
```

⇒ ["Dog", "Cat", "Rabbit"]

Note:- [1,2,3[5,6]] - is array ko flat karna hai array ke under array nahi hona chahiye isliye flat(), or Flatmap() methods use karte hai (nested array)

depth: this specifed how deep a nested array structure should be flattered.

3) flat ()

This method is used to create a new array with one sub-array element concatinated.
↳ link together

ex.

```
Const array1 = [1, [2, [3, [4, 5]]]]
Const nestedArray = array1.flat(); ← defalt 1
```
⇒ [1, 2, [3, [4, 5]]]

```
Const nestedArrayInfinity = array1.flat(Infinity);
Console.log (nestedArrayInfinity);
```
⇒ [1, 2, 3, 4, 5]

4) flatMap() Callback function

This method maps all array elements and creates a new flat array and it's create a new array from calling a function for every array element. It is identically to a map() folled by a flat() depth.

ex.

```
Const array = [1, 2, 3, 4];
array.flatmap((x) => [x * 2]);
```
⇒ [2, 4, 3, 8] ← Callback function

* Searching methods in array

1) indexof() ← SearchElement

This methods of array returns the first index at which a given element can be found in the array, if it's not present then returns -1.

Note :- Don't use typeof array is always gives object
Use only Array.isArray() : it's gives boolean value.
if it is then true and if it's not then false.

ex

```
Const numberfind = [1, 2, 3, 4, 5, 6]
                    0  1  2  3  4  5
Console.log (numberfind.indexof (1));
```

=> 0 because 1 is at index 0 position

```
Consde.log (chamber-find.indexOf (10));
```

=> -1 because 10 is not present in array.

2) includes ( ) → search element

This method of array to determines whether an array includes a certain value among, returns true, if not then retuns false.

ex.

```
Const SearchNumber = [1, 2, 3]
Console.log (searchNumber.includes (2));
```

=> true;

```
console.log (searchNumber.includes (9));
```

=> false

* Array.isArray ( )

This is the static method to determines whether the passed value is an array or not, it's retuns boolean value, true or false.

ex.

```
Console.log (Array.isArray ([1, 2, 3]));
```

=> true

```
Console.log (Array.isArray ('[]'));
```

=> false

Note:- function and callback are same bs just a fancy words.

Date
Page

08- array methods

array of objects

```
Const orders = [
  { dish: "Pasta Carbonara", Price: 14, Spicy: false, qty: 2 },
  { dish: 'Dragon Ramen', Price: 12, Spicy: true, qty: 1 },
  { dish: "Caesar salad", Price: 9, Spicy: false, qty: 3 },
  { dish: "Inferno Wings", price: 11, Spicy: true, qty: 2 },
  { dish: "Truffle Risotto", price: 18, spicy: false, qty: 1 },
]
```

- Data ko smjo sabse pehle. (imp)
1) dish ek string hai
2) price number hai
3) spicy boolean hai
4) qty bhi number hai

ab ispe loop lagao, orders ke upper foreach loop lagao. 1) ForEach ()

// ForEach loop apne under callback leta hai

```
orders.forEach( order , index ) => {
                  ↑ order of element
                          ↑ index of the element
Console.log('#${index+1}: ${order.qty} x ${order.dish}');
```

Kyuki array ka index 0 se start hota hai

access order qty

acess order dish

Output:-

#1 : 2x Pasta Carbonara
#2 : 1x Dragon Ramen
#3 : 3x Caesar Salad
#4 : 2x Inferno Wings
#5 : 1x Truffle Risotto

Note:- forEach loop me aap 3rd parameter bhi le sakte ho jese array or use upon kehte hai.

arrow function

ex.

```
Const myData = orders.forEach((order, index) => {
Console.log(`#${index+1}: ${order.qty}x ${order.dish}`);
});
Console.log(myData);
```

⇒ undefined

(because forEach ka default kaam return in nahi karta hai)

2) map()

This is the method for arrays, it used to create a new array by calling a provided callback on every element of the original array.

callback

```
ex. //restaurant ka bill payment karo
Const receiptlines = orders.map((o) =>
`${o.dish}: $${o.price * o.qty}`);
```

1st element ko jo chahe wo bola kar? farak nahi padta ye aapke internal name hai or scoped hai

```
Console.log(receiptLines);
```

Output=

```
[ 'Pasta Carbonara: $28',
  'Dragon Ramen: $12',
  'Caesar Salad: $27',
  'Inferno Wings: $22',
  'Truffle Risotto: $18',
]
```

Note: difference between forEach() and map()

forEach() returns undefined,

map() returns new array

Filter means condition dete ho ki condition kya hai uske pehle new array print hota hai.

3) filter ()

This method is used to creates a new array containg only the elements from the orginal array that pass specific condition using callback function

ex.

```
Const spicyOrders = orders.filter ((o) => o.spicy);
Console.log (spicyOrders);
```

→ [
{ dish: 'Dragon Ramen', price: 12, spicy: true, qty: 1 },
{ dish: 'Inferno Wings', price: 11, spicy: true, qty: 2 }
]

(it's returns array of object and only print true values because it's matches the condition)

4) reduce ()

reduce ko total calculate karne ke liye use karte hai jese shopping cart, total revenew

syntax

```
reduce((accumulator, currentvalue) =>
    accumulator + currentValue, initialValue,);
```

← callbackfn

currentValue → each element items

- It's manupulate's the data, you can also initialise the value, otherwise it used initialvalue 0 because array ~~start~~ index start's from 0 :)
- Single line me callback ya () apne aap value return karte hai
- {} ise use karne se aap khud value return karvaoge aap ki jimmedari hai, ise explisit return kehte hai

remember: array ke under jitne bhi Object diye hai un sabke pass ek-ek bar line-by-line jana hai yahi methods ka kam hai

Syntax example.

ex, ek value nikalke lana — currenty O — item's

```
const totalRevenue = Orders.reduce((sum, order) =>
{ return sum + order.qty * order.price; }, 0);
Console.log(totalRevenue);
```

=> 107

Reduce example.

ex,

```
Const grouped = orders.reduce((acc, order) => {
  Const category = order.spicy ? "spicy" : "mild"
  }, { spicy: [], mild: [] })
```

← object in key-value pair

empty array

ye initialvalue hai or initialvalue datatype hota hai or aap ise kuchh bhi datatype de sakte hai

accumulator ki initialvalue

acc = { spicy: [], mild: [] } ---- internaly yesa hi hota hai

```
acc[category].push(order.dish);
return acc;
```

*karke element ko access nahi karna chahiye is case me.

```
Console.log(grouped);
```

output=

```
{
  spicy: ['Dragon Ramen', 'Inferno Wings'],
  mild: ["Pasta Carbonara', 'Caesar salad', 'Truffle Risotto']
}
```

Note: reduce() value ko start se access karta hai
reduceRight() value ko last se access karta hai

gotchas - "tricky behaviors"

- Sort()

It's coorks well for strings, arranging the alphabetically.

ex

```
Const fruits = ["Banana", "Apple", "Cherry"]
fruits.sort();
Console.log (fruits);
```

⇒ ['Apple', 'Banana', 'Cherry']

But for number it gives incorrect results and treats them as strings.

that's ~~use~~ why always use compare function.

1) for ascending order ~~(a-b)~~ (a,b) ⇒ a-b;

2) for descending order (a,b) ⇒ b-a;

ex

```
Const ticketNumber = [100, 25, 3, 42, 8];
Const sortdW = [...ticketNumber].sort((a,b)=>a-b);
```

↓ Spread Operator for deep copy

```
Console.log (sortdW);
```

⇒ [3, 8, 25, 42, 100]

Interview question:

forEach()

- forEach() expects a synchronous function, it does not wait for promises.
- There is no way to stop or break a forEach() loop other than by throwing an expection, warna crash ho jaega isme await or break kuchh kam nahi karta.

ex.

```
Const KitchenOrders = [
{dish: "Pasta Carbonara", price: 14, spicy: false, qty: 2},
{dish: "Dragon Ramen", price: 12, spicy: true, qty: 1},
{dish: "Caesar Salad", price: 9, spicy: false, qty: 3},
{dish: "Inferno Wings", price: 11, spicy: true, qty: 2},
{dish: "Truffle Risotto", price: 18, spicy: false, qty: 1},
{dish: "Ghost Pepper Soup", price: 15, spicy: true, qty: 1},
];
```

negative condition ko negation bolte hai

```
Const mildReport = KitchenOrders
.filter (order => !order.spicy)
.map (order => ({
        dish: order.dish,
        total: order.price * order.qty
    }))
```

chaining

hamesh true value pass karta hai to condition true likhiye

.toSorted

() - matlab ek hi line me likhna

({}) - multiple lines me likh sakte ho.

- returns a copy of an array with it's elements sorted
- or yaha bhi aapko no. ke liye (a,b) => a-b lagana padega.
- toSorted jada use kiye jate hai.

0.9 - Objects

```
Const hero = {
        name : "Luna the Brave",
        class : "Mage",
        level : 12,
        health : 85,
        mana : 120,
        isAlive : true,
        };
```

Objects ke under kuchh bhi access karna hai to do hi type hote hai

1) hero.
- agar aapko pata hai ki value yahi hai kahi dynamically nahi aa rahi hai to dot (.) hi prefer karo.

2) Kero[ ]
- agar responce aa raha hai ya ~~data type~~ ~~key~~ accumulator aa raha ye ~~or~~ or agar aap processing kar k rahe ho data ke upper to ye use karo.
isme safty hoti hai agr space hoga to bhi chal jata hai. ex hero["user name"]

- Object me ek to property's add karoge ya to delete karoge.

1) add karna :-
ex, hero.weapon = "fire"

2) delete karna :-
ex, delete hero.level

How to copy past code in browsers console
- type allow pasting in console then it's work.

```
Const ranger = {
        name : "Lakshya the swift",
        agility : 80,
        stealth : undefined
      }
```

Console.log ("name" in ranger); ⇒ true
Console.log ("stealth" in ranger); ⇒ true
Console.log ("tostring" in ranger); ⇒ true

↑
ye prototypal chaining hai ye values ko inheritance karta hai.

To reliability ke liye hum hasownproperty use karte hai

Console.log (ranger.hasOwnProperty ("tostring"))
⇒ false

- .hasOwnProperty ()
this method is used to check if an object has a specific property as its own property

10- Object Methods

```
Const artifact = {
    Keys → name: "Obsidian Crown",   ← values
           era: 'Ancient',
           value: 50000,
           material: 'volcanic glass',
         };
```

How to access all the keys in objects

```
Const keys = Object.keys (artifect);
Console.log (keys)
```

⇒ ['name', 'era', 'value', 'material']

- Object.keys ()

This method returns an array of a given objects own enumerable string-keyed property names.

For values

```
Const values = Object.values (artifect);
Console.log (values)
```

⇒ ['Obsidian Crown', 'Ancient', 50000, 'Volcanic glass']

- Object.values ()

This method returns an array of a given object's Key's ~~values~~ property values.

for entries

```
Const entries = Object.entries(artifect);
console.log(entries);
```

=> [

```
['name', 'Obsidian Crown'],
['era', 'Ancient'],
['value', 5000],
['material', 'Volcanic glass']
```

]

arrays of array

- Object.entries()
This method retains an array of ~~object~~ given object in [key, value] pairs.

- Objects me mostly forof loop hi use hote hai

```
for (const [key, value] of Object.entries(artifect)) {
  console.log(`${key}: ${value}`)
}
```

forof se object se array of array banaya.

output:-

name: Obsidian Crown
era: Ancient
value: 50000
material: volcanic glass

- is tarh se object ko arrays me convert karte hai

array of array = entries = same
entries = from entries = convert in to object

- array ko object me convert karna.

```
Const priceList = [
        ["Obsidian Crown", 50000],
        ["Ruby Pendant", 30000],
        ["Iron shield", 5000],
    ];

Const priceObject = Object.fromEntries (priceList);
Console.log (priceObject);
```

output :-

{'Obsidian Crown': 50000, 'Ruby Pandant': 30000, 'Iron shield': 5000}

- Object.fromEntries ()
This method transfroms a list of key-value pairs.

Interview question
ex.

```
Const displayCase = {
                artifact : "Obsidian",
                location : "Hall A, Case 3",
                locked : true,
            };

Object.freeze (displayCase);
delete displayCase.locked;
displayCase.newprop = "test";
Console.log (displayCase);
```

↓ freeze
↓ it doesn't allows you to change or delete any element.
→ prints same object

```
Const catalogEntry = {
                    id: "ART - 001",
                    description: "Ancient Crows",
                    verified: true,
                };
```

Object.seal (catalogEntry);

⇒ prints the same object

So, what is the differance between Object.freeze() and Object.seal()

- seal is used when you allow edit to existing property, but structural change nahi ho sakte matlab ki new add hoga nahi, kuchh delete bhi nahi hoga sirf edit ho sakta hai

- freeze aapko koi bhi modification nahi karne dega

Interview question

Const secureArtificates = { name : "Ruby Pandant" };

```
Object.defineProperty ( secureArtificate, "catelogId", {
        value : "SEC - 999",          → catelogId ki value
        writable : false,             → ise change nahi kar sakte
        enumerable : true,            → agar default loop karne chahte ho to true hoga tabhi ho sakta hai, agar false hoga to nahi ho sakta hai
        configurable : false,         → matlab ise delete or redefine kar sakte ho kya
    });
```

Note :- ~~Object me har~~ har object loopeble nahi hota hai

Object.defineProperty() :- This method defines a new property directly on an object or modifies an existing property and retuns the object

Date
Page

or

Const SecureArtificats = { name: "Ruby Pendant",
catelogId: "SEC-999" }

godmod hai

Note - dono tarike same hai sirf jab object.defineproperty use karte hai tab hame writable, enumerable or configurable ka access milta hai

Console.log (SecureArtificats.catelogId);
=> SEC-999
// value change karna
secureArtificats.catelogId = "HACKED";
Console.log (secareArtificate.catelogId);
=> SEC-999

Note - value change nahi hai kyuki vo silently fail ho gai kyuki hmne object.define property me writable: false. hai

// for of loop lagao

for (Const [key, value] of Object.entries (secareArtifates)) {
Console.log (`${key}: ${value}`);
}

Output :
name : Ruby Pendant
catelogId : SEC-999

Note - kyk kyuki enumerable : true tha to loops kar sakte hai agar enumerable : false hota to catelogId: SEC-999 print nahi hota vo silently fail ho jata.

ab catelogId ki details nikalo

```
const desc = Object.getOwnPropertyDescriptor
              (secureArtificates, "CatelogId");
Console.log(desc);
```

output:

```
{
    value: 'SEC-999',
    writable: false,
    enumerable: false,
    configurable: false,
}
```

- Object.getOwnPropertyDescriptor()
This method returns the property descriptors of an object.

- Property sabke sat associate hoti by by default ~~values~~ property

```
{
    value: 'Ruby Pendant',
    writable: true,
    enumerable: true,
    configurable: true
}
```

loop key points

1) for ()

- This is the most common loop it is used when you know exactly how many times you want to run the code

Syntax

```
for (initialization; condition; increment) {
    // code
}
```

ex.

```
for (let i=1; i<=5; i++) {
    console.log (i);
}
```

⇒ 1
2
3
4
5

Note: for loop sabse optimise loop hota hai, jitna optimisation ispe lagaya ja sakta, aapke V8 engine ya kisi bhi language pe isi pe lagaya ja sakta hai, kisi large array ko loop karna ho, classic loop ko use karo performans acchhi rehti hai.

2) while ()

The while loop is the tool for when you aren't sure how many times you' need to run code. agar condition true hai to code chalega nahi to turant terminate ho jayega.

Syntax

```
while (condition) {
        // code
}
```

ex.

```
let count = 2;

while (count >= 0) {
    console.log("counting down: " + count);
    count--;
}

console.log("Blast off!");
```

output =

counting down: 2
counting down : 1
counting down : 0
Blast off!

3) do...while ()

it's guarateed to run code at least once, regardless of condition is true or false.

syntax:

```
do {
    // code
} while (condition)
```

ex

```
let password;
do {
    password = prompt("enter password");
} while (password !== "kukumuku123");
alert("Access Granted!");
```

4) for...in

(ise objects pe use karo, arrays pe to avoid karo)

It used to iterate over propeties (keys) of an object.

Syntax

```
for (let key in object) {
    // code
}
```

ex.

```
Const project={
    title: "page",
    color: "white",
}
for (let key in project) {
    console.log (key + ":" + project[key]);
}
```

output

```
title: page
color: white
```

5) for...of

(sada bahar sabpe chalta hai)

The for..of loop creates a loop iterating over iterable objects. This includes built in type like array, string and even object arguments.

Syntax

```
for (variable of iterable) {
    //code
}
```

ex.

```
Const tools = ['Hammer', 'Screwdriver', 'Wrench'];
for (const tool of tools) {
        Console.log (tool);
    }
```

output

⇒ Hammer, Screwdriver, Wrench

11. functions

How to write function

- function is a reusable block of code designed to perform a specific task.

ex

~~function~~

```
Console.log (brewPortion ("Healing Herbs", 3))
function brewPortion (ingredient, dose) {
return 'Brewing potion with ${ingredient}
                (x${dose}) ... Potion ready'
```

ye code is liye execute hega kyki hoisting allow karti hai function ko defined hone se pehle use kar sakte ho.

output

Brewing potion with Healing Herbs (x3) ... potion ready

function expression

```
const mixElixir = function (ingredient) {
    return 'Mixing elexir with ${ingredient}'
}
```

↑↓ ye dono same hi hai, sirf variable name change hai

```
const distilEssence = (ingredient) => {
    return 'Mixing elexir with ${ingredient}'
}
```

- iske pass this nahi hota hai or arguments object bhi nahi hote hai arrow function ke pass

Difference between argument and arguments object.

- Argument

arguments are the values assigned to the function's parameters during the function call

ex.

```
myfunction ('apple', 'banana')
```

→ individual arguments

- arguments Object

the arguments object is an atomatic local variable which is available inside every function only not in arrow function that stores all the arguments passed to that function

ex.

```
arguments[0] // first argument
arguments[1] // second argument
```

Note:- error aana buri bat nahi par program crash hona bura hai. Wahi problem hai tab try catch use karo.

Date
Page

```
function oldBrowsingLogs () {
  Console.log ("Type: ", typeof arguments);
  Console.log ("Is Array:", Array.isArray(arguments));
  console.log (arguments);
  }
oldBrowsingLogs ("Sage", "Rosemary");
```

Output:

→ Type: object (typeof argument is object)

→ Is Array: false (But it's not array)

→ [Argament] {'0': 'Sage', '1': 'Rosemary'}

Convert into array

```
Const argsArray = Array.from (arguments);
Console.log (argsArray);
```

Output

['Sage', 'Rosemary']

function ke sideeffect pure in pure impure function

```
let globalCount = 0;
function breaCount(name) {
    globalCount++;
}
```

→ iski value change ho rahi hai esi koi function ke side-effect kehte hai

- ye function jo globalCount ko change value, ko change kar raha hai isi kahi impure function kehte hai
- It's depend on external state, value ko mutate kar raha hai

function side-effect :- apne scope ke bahar values ko change karna

- ek function ko technically pure tab bola jata hai jab vo function bahar ki koi value ko change na kare, apne under hi jo kam karna hai vo kare

IIFE - Immediately Invoked function Expression

- ek function banaya use turant execute kardiya

```
()()
```

- It runs as soon as it is defined.

Standard IIFE

```
(function () {
        // Standards ...
})();
```

ex.

```
()()
(function () { })():
const postionshop = (function () { })()
```

} all are same

- function se aap jo chahe vo returns karva sakte ho jese string na ya number, function se function bhi return kar sakte ho.

Note: Callback fancy name of function

Date
Page

High Order function
= function ke under function

High-order functions are functions that operate on other functions by either taking them as arguments (callbacks) or returning them as results

ex.

```
function anotherfunctionforClass (brewAndCount) {
  return function newBrew () {
                    // code
                  }
              }
```

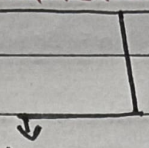
IIFE example

```
const potionshop = (function () {
    let inventory = 0

  return {
      brew () {
          inventory ++
          return 'Brew potion #${inventory}
        },
        getStock () {
          return inventory }
      }
  }
})()
console.log (potionshop)
```

Outpat

{brew: [function :brew], getstock: [function: getstock]}



function defination

```
console.log(portionshop.brew());
```

⇒ Brew portion #1

```
console.log(portionshop.inventory);
```

⇒ undefined.

(IIFF ko memory ka refference nahi hota)

Closure

A closure is the combination of a function bundled together with references to its surrounding state.

(memory ka sat me refference leke aate hai)

ex.

```
function makFunc() {
    const name = "Mozila";
    function displayName() {
        console.log(name);
    }
    return displayName;
}
const myfunc = makeFunc();
myFunc();
```